B.Tech. Minor Project Report

on

Design and Development of an Agentic Retrieval-Augmented Generation Framework for Automated Legal Case Analysis

by

Samudraneel Sarkar (20233249)
Patil Sakshi (20233209)
Nikhil Kumar (20233202)
Manpreet Kaur (20233180)

A Report Submitted
in Partial Fulfillment of the Requirements
for the award of the Degree of
Bachelor of Technology
in
Computer Science & Engineering



COMPUTER SCIENCE AND ENGINEERING DEPARTMENT
MOTILAL NEHRU NATIONAL INSTITUTE OF TECHNOLOGY
ALLAHABAD, PRAYAGRAJ

November 2026

UNDERTAKING

I declare that the work presented in this report titled "*Design and Development of an Agentic Retrieval-Augmented Generation Framework for Automated Legal Case Analysis*", submitted to the Computer Science and Engineering Department, Motilal Nehru National Institute of Technology Allahabad, Prayagraj, for the award of the ***Bachelor of Technology*** degree in ***Computer Science & Engineering***, is my original work. I have not plagiarized or submitted the same work for the award of any other degree. In case this undertaking is found incorrect, I accept that my degree may be unconditionally withdrawn.

	Samudraneel Sarkar	20233249
	Patil Sakshi	20233209
Place: Prayagraj	Nikhil Kumar	20233202
Date: November 2026	Manpreet Kaur	20233180

CERTIFICATE

Certified that the work contained in the report titled "*Design and Development of an Agentic Retrieval-Augmented Generation Framework for Automated Legal Case Analysis*", by *Samudraneel Sarkar (20233249), Patil Sakshi (20233209), Nikhil Kumar (20233202), and Manpreet Kaur (20233180)*, has been carried out under my supervision and that this work has not been submitted elsewhere for a degree.

November 2026

Dr. Vibhav Prakash Singh
Computer Science and Engineering Dept.
MNNIT, Allahabad

Abstract

Automated analysis of legal cases in order to find applicable judgements and precedents is a promising application of large language models. However, in law, every claim or argument must be traceable to an authoritative legal document and LLMs often fall prey to hallucination and fabricated language generation. We investigate how a retrieval augmented generation system should be designed to handle legal work in realistic settings where case descriptions run into several thousand tokens and output must cite statutes and precedents that are directly relevant. We quantitatively examine RAG pipelines and different retrieval, embedding and chunking strategies and evaluate their performances on the IL-PCSR benchmark, which evaluates statutes and precedents over Indian case judgements. We also validate our pipeline on BSARD as external validation and establish three findings. First, the two retrieval tasks reward different machinery: lexical retrieval dominates precedent finding, whereas long context dense retrieval, oftentimes combined with lexical retrieval by rank fusion, dominates statute retrieval. Second, compression of any form of the case, including expansion, rhetorical restriction, truncation for the cross-encoder re-ranker, etc., degrades retrieval quality. Third, a re-ranker improves the ranking only when relevant signal is semantic and the re-ranker can read the complete query, and otherwise a truncated cross encoder leads to worse performance. These findings produce an asymmetric optimal architecture with a dense-lexical statute leg along with language model reranking, and a purely lexical precedent leg. We then test out a bounded agent loop on top of this pipeline in order to place guardrails like retry upon failure, external fallback mechanism and mechanical validation of citations, and evaluate the resulting architecture on an expanded corpus of 28,074 section-level statute documents covering 302 Central acts.

Acknowledgements

We express our sincere gratitude to our supervisor, Dr. Vibhav Prakash Singh, Department of Computer Science and Engineering, Motilal Nehru National Institute of Technology Allahabad, for his guidance, help, and constructive feedback throughout this project.

We also thank the faculty and staff of the Department of Computer Science and Engineering for the environment and the resources that made this work possible. We also thank the authors of the IL-PCSR and BSARD corpora for their invaluable contributions in releasing the datasets with reproducible baselines, without which a fair comparative study could not have been conducted. We also sincerely thank our friends at NUJS Kolkata for their inputs on matters which require a legal perspective.

Finally, we thank our families and friends for their constant support and love during the course of this work.

Contents

Abstract **iv**

Acknowledgements **v**

1 Introduction **1**

1.1 Background 1
1.2 Motivation 2
1.3 Problem Statement 3
1.4 Research Objectives 4
1.5 Scope of the Study 5

2 Literature Review and Theoretical Foundations **6**

2.1 Legal Informatics and the Case Analysis Problem 6
2.2 Language Models and the Verifiability Problem in Law 7
2.3 Retrieval-Augmented Generation 7
2.4 First-Stage Retrieval 8
2.4.1 Lexical Retrieval 8
2.4.2 Dense Retrieval 8
2.4.3 Hybrid Retrieval and Fusion 8
2.4.4 Second-Stage Reranking 9
2.5 Query Processing 9
2.6 Agentic Workflows 9
2.7 Research Gaps 10

3 Methodology and Experimental Investigation **11**
3.1 The IL-PCSR Benchmark . . . 11
3.2 Task Formalisation . . . 12
3.3 Metrics and Evaluation Protocol . . . 12
3.4 The Experiment Harness . . . 12
3.5 Experimental Design . . . 13
3.6 First-Stage Results . . . 14
3.7 Fusion Results . . . 15
3.8 Reranking Results . . . 15
3.9 Query-Processing Results . . . 16
3.10 Agent Benchmark Results . . . 17
3.11 Learned Sparse and Late-Interaction Results . . . 18
3.12 Comparison with the Published Baseline . . . 19
3.13 External Validation on BSARD . . . 21
3.14 Synthesis of Findings . . . 22

4 Design and Architecture of the Agentic RAG Framework **24**
4.1 From Findings to Architecture . . . 24
4.2 System Overview . . . 25
4.3 The Retrieval Substrate . . . 25
4.3.1 Document Representation . . . 25
4.3.2 The Lexical Leg . . . 26
4.3.3 The Dense Leg . . . 26
4.3.4 Fusion and Reranking . . . 26
4.4 The Agentic Control Loop . . . 27
4.5 Expanded Statutory Corpus . . . 29

5 Implementation and Application Development **31**
5.1 Technology Stack . . . 31
5.2 The Evaluation Harness . . . 31
5.3 The Agent Implementation . . . 32
5.4 The Serving Application . . . 32

5.5 End-to-End Request Flow 33

6 Results, Evaluation, and Discussion **34**

6.1 Evaluation Framework 34

6.2 System-Level Behaviour 34

6.3 Live Verifications 35

6.4 Interpretation 37

6.5 Limitations and Threats to Validity 38

7 Conclusion and Future Work **39**

7.1 Conclusion 39

7.2 Findings in Relation to the Research Objectives 40

7.3 Future Work 40

References **42**

Chapter 1

Introduction

1.1 Background

The documents a legal system produces fall into two broad classes. Statutes are acts of the legislature: prescriptive texts organised hierarchically into chapters, sections, and sub-sections, each section stating a rule, a definition, an offence, or a remedy in compact formal language. The Indian penal code, its successor the Bharatiya Nyaya Sanhita, and the procedural and evidence codes are examples. Case law is the second class: a judgment records the facts of a dispute, the issues framed, the reasoning of the court, the ratio decidendi that binds future courts, and the final order. A professional query to such a corpus is rarely a keyword; it is a narrative of facts from which the applicable sections and the controlling precedents must be identified, a task this work treats as two distinct retrieval problems following the IL-PCSR benchmark [1].

Large language models are neural networks trained to predict the next token over very large text corpora. Their fluency comes from parametric memory: the facts a model knows are encoded implicitly in its weights rather than stored in any document it can point back to, and the model will produce a plausible continuation whether or not the underlying fact is true. Two consequences matter for law. First, hallucination: a model can emit a confident but non-existent section number or case citation. Second, the context window: a model can only attend to a bounded span

of text, which a long judgment or a detailed case description can exceed.

Applications of these models to law now span judgment summarisation, document drafting, question answering, and retrieval, and dedicated benchmarks measure them: LegalBench collects a broad suite of legal reasoning tasks [2] and IL-TUR covers Indian legal text specifically [3]. Long-input encoders such as LawFormer extend the context budget for legal documents [4], while specialised encoders such as SAILER are trained for case retrieval [5]. What these applications share is a dependence on retrieval: the answer a legal system gives is only as trustworthy as the documents it was built from, which is the central concern of this study.

1.2 Motivation

India administers one of the largest judicial systems in the world. The National Judicial Data Grid records tens of millions of pending matters across the district and subordinate courts [6], and behind each matter lies a recurring research task: given a set of facts, identify the statutory provisions that govern it and the prior judgments that bear upon it. Legal professionals perform this task against a corpus that is large, formally phrased, and constantly amended; a single case description can run to several thousand tokens before a single authority is consulted. Any reduction in the effort required to locate and verify the applicable law has direct practical value.

Large language models appear to answer this need, and their fluency is precisely the problem. A model asked for the punishment for murder produces a confident section number; asked for the controlling precedent, it produces a confident case name. Neither answer carries any mechanism by which a reader can distinguish a true citation from a plausible fabrication, and in a domain where a wrong citation misleads counsel rather than merely a casual reader, fluency without verifiability is a liability. The requirement that follows is not better generation but grounding: every claim the system makes must be produced from, and checkable against, retrieved legal documents.

Retrieval-augmented generation supplies the grounding mechanism [7]. Whether

it suffices for legal case analysis is, however, an empirical question rather than an assumption. The queries are not keyword strings but complete case descriptions averaging roughly 4,500 tokens; the corpora are of two kinds whose notions of relevance differ; and the retrieval machinery deployed in most RAG systems is designed and validated for short queries. Which components of that machinery transfer to long legal queries, which do not, and what an agentic controller contributes beyond a fixed pipeline, are the questions this dissertation answers through controlled measurement followed by system construction.

1.3 Problem Statement

The task is automated legal case analysis over Indian law, formalised as follows.

Input. A natural-language case description q of several thousand tokens, comprising a factual narrative, the issues framed, and the procedural history of the matter, in the form used by the IL-PCSR benchmark [1].

Output. Three artefacts: (i) an ordered set S of statutory provisions drawn from a statute corpus, (ii) an ordered set P of precedent documents drawn from a precedent corpus, and (iii) a structured dossier D containing a direct answer, the applicable provisions, a grounded analysis, and recommended steps, such that every substantive claim in D is supported by an element of $S \cup P$ and carries a verifiable citation.

Functional requirements.

R1 Grounding. Every emitted claim must cite a document present in the retrieved evidence set; unverifiable identifiers are suppressed mechanically. When retrieval cannot supply adequate evidence, the system must disclose the insufficiency and invoke an external fallback rather than present ungrounded material as retrieved authority.

R2 Sufficiency awareness. The system must determine whether the retrieved evidence genuinely addresses the case, independently of retrieval scores, which cannot distinguish a correct statute from a lexically similar incorrect one.

R3 Bounded recovery. Insufficient evidence must trigger a bounded re-retrieval, and persistent insufficiency must route to an external source explicitly labelled as not corpus-grounded.

The problem is difficult for three reasons. First, the input is long: an IL-PCSR case description averages roughly 3,600 words, which exceeds the input budget of standard embedding models and cross-encoders, so part of the information need is invisible to them by construction. Second, the two output corpora embody different notions of relevance: statutes are relevant doctrinally, by mapping facts to the elements of a provision, whereas precedents are relevant factually, by similarity of situation and vocabulary. There is no reason to expect a single retriever to be optimal for both. Third, the correctness constraint is strict: a system that emits a plausible but incorrect citation has failed materially, not marginally.

1.4 Research Objectives

1. To establish, through controlled experiments on a public Indian legal benchmark, which retrieval, fusion, reranking, and query-processing mechanisms are effective for long-form statute and precedent retrieval.

2. To derive from those measurements a retrieval architecture in which each component is justified by evidence rather than by convention.

3. To design an agentic control loop that adds to the fixed pipeline the guarantees it lacks: evidence-sufficiency judgment, bounded recovery, citation validation, and labelled external assistance.

4. To implement the framework as a working application over an expanded Indian statutory corpus, and to verify its behaviour on queries the benchmark itself cannot answer.

5. To examine whether the derived architecture and its design rules transfer to other legal datasets and jurisdictions.

1.5 Scope of the Study

The study addresses retrieval and grounded synthesis for Indian central legislation and case law in English. It does not attempt legal judgment prediction, statutory interpretation as a normative exercise, drafting of legal instruments, or jurisdictions outside India. Benchmark evaluation is conducted on the IL-PCSR development split; the published test-split figures of the benchmark are cited under their own protocol and are not equated with development-split figures. Language-model calls in the system use externally hosted models.

Chapter 2

Literature Review and Theoretical Foundations

2.1 Legal Informatics and the Case Analysis Problem

Computational assistance for legal research predates neural methods. Early systems relied on the vector space model [8] and on term-weighting schemes [9]; the probabilistic framework that succeeded them produced BM25 [10], which remains the standard lexical baseline decades later. Two properties of legal text shaped this line of work: legal meaning is carried by phrases rather than isolated terms, and the documents are long. Statute identification in particular has been approached through multi-label classification [11, 12] and through graph-based methods such as LeSICiN, which models statute hierarchy for Indian documents [13, 14]. Prior case retrieval has been modelled as ranking from the outset, because the candidate pool is large and relevance is similarity-based [15].

Indian legal text presents a specific structural property on which part of this study depends: judgments are not segmented into facts, issues, arguments, and rulings by any reliable marker. The rhetorical-role literature supplies automatic segmentation instead. The ILDC corpus established sentence-level legal segmentation for Indian judgments [16]; Malik et al. labelled rhetorical roles such as Facts, Issue,

and Court Reasoning [17]; and DeepRhole provides a trained classifier for the same labels [18]. The IL-PCSR queries used in this study carry exactly these segment annotations, which makes a rhetorical-restriction ablation directly executable.

2.2 Language Models and the Verifiability Problem in Law

Large language models have been evaluated across legal tasks by LegalBench [2] and, for Indian law specifically, by IL-TUR [3]. A consistent finding across this literature is that model fluency is not accompanied by source fidelity: models produce provisions and case names that read correctly and do not exist. Domain adaptation has been attempted through pretraining on legal corpora; SAILER trains a structure-aware encoder for case retrieval [5], and LawFormer extends the input budget for long legal documents [4]. The present study measures rather than assumes the benefit of such models on the task at hand, and its results for unmodified legal-domain encoders are negative.

2.3 Retrieval-Augmented Generation

Retrieval-augmented generation conditions a generated answer on documents retrieved at inference time [7]. The architecture transfers the burden of factuality from the generator's parametric memory to the retriever's index, which is auditable. Its performance is therefore bounded by retrieval quality, and the design space of that retriever is the subject of the next three sections.

2.4 First-Stage Retrieval

2.4.1 Lexical Retrieval

BM25 scores a document against a query by inverse-document-frequency-weighted term overlap, with term-frequency saturation and document-length normalisation [10]. Three of its properties matter to this task. It is deterministic, so its rankings are fully auditable. It rewards phrase overlap when indexed over word n-grams, which matches how statutory and judicial language encodes meaning. Its cost does not grow with query length, since additional query terms only refine the match. The implementation decision that dominates its behaviour on legal text is the tokenisation unit: word n-grams preserve multi-word terms of art such as *culpable homicide* or *unlawful assembly* that unigram indexing fragments [19].

2.4.2 Dense Retrieval

Dense retrievers map queries and documents into a shared vector space with a bi-encoder and rank by similarity [20, 21]. Their advantage is semantic generalisation: a document can be retrieved without sharing any vocabulary with the query. The corresponding risk is that the match is not phrase-verifiable, and two constraints govern their use on this task. The first is context length: a 512-token encoder reads roughly a seventh of an average IL-PCSR case description and ranks on that fragment alone, while longer-input models such as the Qwen3-Embedding family [22] read several times as much. The second is task fit: a legal-domain encoder pretrained for one retrieval objective does not automatically transfer to the matching of case descriptions to provisions.

2.4.3 Hybrid Retrieval and Fusion

Lexical and dense retrievers fail in complementary ways, which motivates fusion. Reciprocal rank fusion combines ranked lists by position alone, assigning each document $\sum_r 1/(k+\text{rank}_r)$ over the contributing retrievers [23]. The method requires no score calibration, but it interpolates rather than amplifies: fusing a strong leg with a

weak leg produces a ranking between the two [24]. Whether this interpolation helps or harms depends on whether both legs carry signal, which Chapter 3 measures per task.

2.4.4 Second-Stage Reranking

A reranker rescores a small candidate pool with a more expensive model. Cross-encoders score each query-document pair jointly [25]; late-interaction models preserve token-level alignment [26]; learned sparse representations combine lexical and neural views [27, 28]; and listwise methods ask a generative model to order the pool directly, an approach whose effect on legal ranking has been measured separately by Ma et al. [29] and by the IL-PCSR baseline itself [1]. All of these share one constraint that decides their behaviour on this task: a reranker that cannot read the full query cannot rank against it.

2.5 Query Processing

The standard interventions on a query are expansion, restriction, reformulation, and truncation. Pseudo-relevance feedback expands the query with terms harvested from the first-pass results [30]; on a corpus whose queries are already long and redundant, its benefit is questionable by inspection and is measured here directly. Restriction keeps only selected rhetorical-role segments, an application of the segmentation literature [17, 18] to retrieval rather than to labelling. Reformulation and summarisation compress the query, the strategy the IL-PCSR baseline itself employs for its summary-conditioned models [1]. Truncation is the implicit intervention of every bounded-context model. Chapter 3 evaluates each.

2.6 Agentic Workflows

An agentic system places a language model in the control loop: the model selects tools, inspects intermediate results, and iterates [31, 32]. Self-reflective retrieval goes further and has the model explicitly assess whether retrieved evidence suffices

before generating [33]. For legal case analysis the candidate agentic contributions are four: a judgment of evidence sufficiency that lexical scores cannot supply, one reformulated retrieval upon failure, a graceful and labelled fallback when the corpus lacks the answer, and mechanical verification that emitted citations exist in the retrieved set. Prior case retrieval research has used event extraction [34], graph neural networks [35, 36], citation structure [37], and judgment-guided retrieval [38]; the IL-PCSR benchmark ties several of these threads together and supplies the evaluation bed for this study [1].

2.7 Research Gaps

Three gaps follow from the literature. First, published legal retrieval evaluations compare retrievers on uniform settings, but the two legal corpora reward different notions of relevance; whether legs should differ per task is measured here rather than assumed. Second, standard query-processing interventions are evaluated for short queries; their behaviour on multi-thousand token case descriptions is an open question with a uniform answer. Third, agentic RAG systems are described qualitatively in the literature; their measurable contribution over a fixed pipeline on a shared benchmark, including citation validity rates, is not. This dissertation addresses all three.

Chapter 3

Methodology and Experimental Investigation

3.1 The IL-PCSR Benchmark

IL-PCSR [1] is a parallel corpus for statute and precedent retrieval over Indian case judgments. It comprises 6,271 expert-labelled case descriptions partitioned into 5,017 training, 627 development, and 627 test queries. Each query is a complete judgment segmented into rhetorical-role spans [17, 39], averaging roughly 3,600 words and 40 segments on the development split, with explicit statute and precedent citations masked by the dataset authors using a legal named-entity recogniser and personal names anonymised for bias control [40].

The two gold corpora differ in nature. The statute pool contains 936 documents, sections drawn from central acts, averaging about 716 words each; the precedent pool contains 3,183 judgments of the Supreme Court and the High Courts, averaging about 7,900 words each. Development queries cite 3.73 statutes and 2.72 precedents on average.

The two retrieval tasks differ fundamentally. Statute retrieval (LSR) requires mapping factual circumstances to governing provisions; the relevance signal is doctrinal. Precedent retrieval (PCR) requires identifying prior cases of similar situation; the relevance signal is factual similarity, expressed largely in shared phrases

and procedural context. The experiments in this chapter repeatedly turn on this distinction.

3.2 Task Formalisation

Given a query q and a candidate pool $\{c_1, \ldots, c_l\}$, with $c_i \in \mathcal{S}$ for LSR and $c_i \in \mathcal{P}$ for PCR, a retrieval model produces a ranked list $\hat{C}(q) = [c_{q_1}, \ldots, c_{q_l}]$ by decreasing relevance score. The gold set for q is the set of candidates actually cited by the judgment, with citation strings masked from the query text. The output consumed downstream is the top-k prefix of $\hat{C}(q)$.

3.3 Metrics and Evaluation Protocol

Retrieval quality is reported as recall@k, precision@k, macro-F1@k, mean average precision (MAP), mean reciprocal rank (MRR), and nDCG@k, computed per query and macro-averaged. All experiments in this chapter evaluate the development split at $k = 10$ unless stated otherwise; the agent benchmark uses a 50-query development slice at $k = 8$.

Two protocol distinctions are stated once and then used throughout. The published IL-PCSR table reports macro-F1 at the best per-query depth on the test split, an oracle over evaluation depths; this study reports fixed-depth F1@10 on the development split. The two protocols are not numerically equivalent: comparisons are therefore stated in terms of which mechanism dominates inside each protocol, never in terms of raw numbers.

3.4 The Experiment Harness

All configurations execute through a shared harness. A JSON specification declares the dataset slice, chunking strategy, retriever, query strategy, and reranker; the runner builds or loads the index, issues every query through an identical code path, and writes per-query metric files, and each run records its git commit, config hash, seed,

and platform in the result file. Compute-bound legs (dense encoding, cross-encoder reranking, and listwise reranking) executed on remote GPU infrastructure, a Kaggle T4 for lighter workloads and a Modal A10G for heavier ones; lexical experiments ran on CPU. The harness's BM25 leg uses BM25S over word n-grams with $k_1 = 1.6$ and $b = 0.7$ [19].

3.5 Experimental Design

The investigation proceeds as a sequence of questions, each answered by a named configuration.

Experiment 1: tokenisation and granularity. Whole-document unigram BM25 against n-gram variants and chunked indexing, establishing the lexical baseline and the effect of document segmentation.

Experiment 2: dense first-stage retrieval. General-purpose embedders (BGE), legal-domain embedders (Legal-Embed, SAILER), and long-context embedders (Qwen3-Embedding-0.6B and 4B), under the context budgets their models impose.

Experiment 3: rank fusion. Reciprocal rank fusion ($k = 60$) over lexical-dense pairs, on both tasks, testing whether fusion helps where constituent legs are and are not competitive.

Experiment 4: second-stage reranking. A cross-encoder (bge-reranker-v2-m3, 2,048-token window) and a listwise language-model reranker (a reasoning model over the fused top-50 pool, 20 numbered snippets of 1,000 characters each), against their first-stage pools.

Experiment 5: query processing. Pseudo-relevance feedback at three settings, restriction to rhetorical-role segments in four configurations, and the truncation and reformulation effects observable across the other experiments.

Experiment 6: the agentic loop. A 50-query benchmark of the route-retrieve-reflect-synthesise agent, measuring retrieval quality of its final evidence union alongside its behavioural statistics.

Experiment 7: learned sparse and late interaction. BGE-M3 in all four of its modes: dense, learned sparse, RRF union, and ColBERT MaxSim rerank over

the union pool.

3.6 First-Stage Results

Table 1 reports the first-stage configurations.

Table 1: First-stage retrieval on the development split ($n = 627$). Best per column in bold; the 5-gram precedent row was measured on the whole-document index required by that tokenisation.

	Statute (LSR)			Precedent (PCR)	
Configuration	F1@10	MAP	MRR	F1@10	MAP
BM25 unigram, whole document	8.39	9.57	20.55	11.02	18.41
BM25, fixed-512 chunks	11.61	12.90	28.92	16.48	19.11
BM25, recursive-512 chunks	11.46	12.87	29.38	17.19	19.75
BM25 2-gram, recursive-512	10.55	12.59	30.77	20.15	27.11
BM25 5-gram, whole document	—	—	—	**20.73**	**39.90**
BGE-base, 512-token	5.67	6.72	14.73	4.05	5.73
Legal-Embed	7.98	9.01	19.35	4.34	6.74
SAILER	0.23	0.12	0.47	0.02	0.04
Qwen3-Embedding-0.6B	12.57	15.16	30.93	11.07	17.18
Qwen3-Embedding-4B	**14.29**	**19.79**	37.38	12.59	19.65

The two tasks exhibit opposite preferences. On precedents the parameter-free lexical leg dominates outright: whole-document 5-gram BM25 approximately doubles the MAP of the best dense configuration, because precedent relevance in this benchmark is factual similarity and factual similarity is carried by shared phrases that n-gram matching reads directly. On statutes the ordering inverts: lexical retrieval is weak, the long-context embedders are strong, and the 4B variant is the strongest single leg measured. The same evaluation ranking dense retrieval last on one task and first on the other is the direct argument for task-conditional legs.

Three subsidiary results are instructive. Every 512-token embedder scores below even chunked BM25 on both tasks, because it ranks documents against a fraction

of the query. SAILER, a legal-domain encoder, scores near zero on both tasks, indicating that pretraining on legal text does not imply competence at this matching task. And chunk granularity is a second-order decision: recursive-512 and fixed-512 differ within noise, while whole-document indexing wins where n-gram phrase context must span the document.

3.7 Fusion Results

Table 2 reports reciprocal rank fusion over the first-stage legs.

Table 2: Reciprocal rank fusion on the development split. The fused pool is formed from each leg's top ranks at $k = 60$.

	Statute (LSR)			Precedent (PCR)	
Configuration	F1@10	MAP	MRR	F1@10	MAP
RRF: BM25 + BGE-base	8.29	9.48	19.97	10.09	15.35
RRF: 5-gram BM25 + Qwen3-0.6B	13.65	17.91	38.91	19.09	29.85
RRF: 5-gram BM25 + Qwen3-4B	**14.68**	**19.12**	**40.53**	19.74	31.60

Fusion improves retrieval precisely where the constituent legs are competitive. On statutes, fusing the lexical and long-context dense legs produces the strongest first-stage ranking measured, exceeding either leg alone, because the legs err in different places and both carry signal. On precedents the same operation draws the dominant lexical ranking toward the weaker dense one and loses ground relative to 5-gram BM25 alone; fusing BM25 with the weak BGE leg degrades statutes as well. The pattern is consistent: fusion interpolates rankings rather than creating signal, so its benefit is conditional on leg quality, not automatic.

3.8 Reranking Results

Table 3 reports the two second stages against their first-stage pool.

Table 3: Second-stage reranking over the fused top-50 pool. The listwise reranker reads the query (up to 4,000 characters) beside 20 numbered snippets.

	Statute (LSR)			Precedent (PCR)	
Configuration	F1@10	MAP	MRR	F1@10	MAP
First-stage pool (RRF c2)	13.65	17.91	38.91	19.09	29.85
Cross-encoder rerank	7.46	6.56	16.14	6.89	7.28
LLM listwise rerank	**14.83**	**25.20**	**50.88**	16.62	25.48

The cross-encoder result is the clearest context-length failure in the study. Truncated to 2,048 tokens, it scores candidates against roughly the first half of each case description and draws both tasks below their first-stage pools; precedent recall@10 collapses from 53.1% to 19.2%, so the ordering it emits is anti-correlated with relevance rather than merely weak. The failure is of context, not of ranking quality: a second stage that cannot read the query ranks a different problem from the one posed.

The listwise reranker shows the mirror image. Because the language model reads enough of the query alongside the candidates, it improves the statute leg substantially, gaining 7.3 MAP points and 12.0 MRR points over its pool and reaching a best-depth F1 of 23.8 at k=2. The same operation reduces precedent MAP by 4.5 points, consistent with the task's phrase-lexical relevance signal being invisible to semantic judgment. Reranking is therefore included on the statute leg and excluded from the precedent leg, each for measured reasons.

3.9 Query-Processing Results

Four interventions, each a defensible attempt to improve a long query, all degraded retrieval (Table 4).

Table 4: Query-processing interventions. Reference rows are the unmodified-query runs on the same index; the PRF statute row scores 6.85 F1@10 against a 10.55 reference.

Intervention	F1@10	MAP
Precedent task (5-gram BM25, filtered index)		
Full query (reference)	20.73	39.90
PRF expansion (d=3, e=10)	19.15	32.49
Facts + Issue segments only	3.86	5.21
Facts segments only	5.93	9.11
Issue segments only	3.73	5.91
All segments except Court Reasoning	18.05	31.55
Statute task (2-gram BM25)		
Facts + Issue segments only	4.63	5.04
All segments except Court Reasoning	7.41	9.56

Pseudo-relevance feedback drifts the query toward whatever the first pass already retrieved, including its errors, and the deviation is near zero across all three expansion settings. Restricting the query to individual rhetorical sections collapses retrieval almost entirely, because the procedural posture and reasoning context carry signal that no single section holds. Removing only the Court Reasoning segments costs about eight MAP points on precedents and is mildly harmful on statutes as well. Truncation was examined in Section 3.8, and reformulation inside the agent benchmark loses the specific vocabulary the gold documents employ. Each mechanism fails differently, and all fail; the serving pipeline consequently issues the case text verbatim.

3.10 Agent Benchmark Results

The 50-query agent benchmark exercises the full loop (Table 5).

Table 5: Agent benchmark, 50-query development slice, BM25 2-gram legs at top-8.

Quantity	Value
Queries evaluated	50
Retrieval calls issued	78
First verdict: evidence sufficient	22
Complete dossiers produced	49 of 50
Citations emitted	167
Citations failing validation	0
Total LLM latency	830.9 s
Wall-clock duration	13.9 min

The call count exhibits the loop in operation: 28 of 50 queries failed their first sufficiency verdict, were re-retrieved once with reformulated queries, and all but one then produced a complete dossier. The citation row states the property a legal tool cannot compromise: no emitted citation required removal. The retrieval-quality side is equally instructive: the agent's evidence union scores below the verbatim single-shot arm (statute MAP 9.5 against 12.6; precedent 7.0 against 27.1), because the router's shortened queries lose the vocabulary that drives lexical retrieval.

3.11 Learned Sparse and Late-Interaction Results

BGE-M3 was evaluated in all four of its modes on both tasks (Table 6).

Table 6: BGE-M3 all-modes on the development split. All four modes score below the corresponding lexical or Qwen3 legs.

	Statute (LSR)			Precedent (PCR)	
BGE-M3 mode	F1@10	MAP	MRR	F1@10	MAP
Dense	7.76	9.24	19.94	9.53	14.68
Learned sparse	10.23	12.32	24.84	11.07	17.05
RRF union	7.84	6.04	13.46	12.02	18.40
ColBERT MaxSim rerank	6.82	8.49	18.50	8.21	11.75

No mode approaches the task champions: the strongest BGE-M3 configuration trails 5-gram BM25 on precedents by roughly twenty MAP points and trails the fused Qwen3 leg on statutes. The late-interaction reranker, which the published baseline does not evaluate, performs worst of all, a second data point for the context-length constraint that governs this task family.

3.12 Comparison with the Published Baseline

The published IL-PCSR table reports, on the test split under the best-k protocol, whole-document 5-gram BM25 as the strongest lexical row and GPT-4.1 two-stage reranking over a fine-tuned ensemble as the strongest configuration evaluated [1]. The strongest non-fine-tuned mechanism in that table is the same class this study ranks strongest on the development split, and the strongest reranker class agrees as well. The differences beyond that shared base are the task-conditional legs, the agentic loop, and the expanded corpus.

Table 7: IL-PCSR rows against the assembled legs. Every block reports the best per-configuration depth; IL-PCSR rows measure the test split, this work the development split.

		Statute (LSR)			Precedent (PCR)		
System	Training	F1	MAP	MRR	F1	MAP	MRR
IL-PCSR, no training (test split, best-k)							
BM25 para-wise, Max-All	none	18.59	21.82	44.32	27.87	38.15	52.32
BM25 5-gram, full doc.	none	16.98	17.88	40.08	33.29	43.98	58.55
This work, no training (dev split, best-k)							
RRF + Qwen3 + LLM rerank	**none**	**23.82**	**25.20**	**50.88**	19.28	25.48	39.23
BM25 5-gram, whole doc.	**none**	—	—	—	**32.19**	**39.90**	**57.28**
IL-PCSR, trained on benchmark (test split, best-k)							
Para-GNN + BM25 ensemble	fine-tuned	39.44	52.13	73.77	38.77	50.21	63.81
+ GPT-4.1 rerank, 2-stage	+ API	46.11	61.06	81.55	43.31	54.43	68.88

Within the training-free tier the assembled statute leg leads on every metric: F1 23.82 at its best depth against 18.59 for the strongest published lexical row, MAP

25.20 against 21.82, and MRR 50.88 against 44.32. The precedent leg sits within roughly a point of the published 5-gram row on all three metrics (32.19, 39.90, 57.28 against 33.29, 43.98, 58.55), the residual expected between development and test splits. The upper rows of the published table, however, share a property the lower rows do not: they are trained on the benchmark itself.

Para-GNN requires rhetorical-role labels for every paragraph, which the benchmark obtains from the source website's annotations, a corpus-specific resource; a per-document graph linking paragraph nodes to a global node through role-labelled edges; and a two-layer graph attention network trained contrastively for one hundred epochs with one positive and 999 negatives per query on an 80 GB accelerator, all of it conditioned on labelled query–candidate pairs for the corpus in question. The ensemble adds a further fitted component, a per-query fusion weight learned by transfer between the two tasks.

Nor is the headline row exempt: the GPT-4.1 figures are produced by reranking the candidate pool of that fine-tuned ensemble, so the strongest published stack depends end to end on trained components.

A live legal corpus invalidates this arrangement. Judgments are delivered and statutes are enacted, amended, or repealed continuously, and each expansion of jurisdiction introduces fresh statute families; every such addition obliges re-segmentation, re-graphing, and eventually re-fitting on new labels. At scales well beyond the benchmark's pools — the expanded statute corpus alone indexes 28,074 documents, against 936 in the benchmark — and on text this syntactically and semantically dense, periodic retraining is not a maintenance task a practitioner can run. The framework therefore commits to legs whose index updates are stateless and incremental: a new document is tokenised or encoded, and no model is re-fitted. For the ever-expanding corpora of ordinary legal work, the training-free configuration is the appropriate design point, and the accuracy headroom that supervision buys, quantified under one protocol, is the first item of future work.

3.13 External Validation on BSARD

The measurements so far describe a single benchmark. To test whether the design rules travel, the statute leg is run unchanged on an unrelated corpus, BSARD [41], a Belgian statutory-article benchmark authored in French: 22,633 articles and 222 jurist-labelled test questions, whose published evaluation reports recall at depth 100 against a series of zero-shot and fine-tuned baselines. The legs, metrics, and protocol of this chapter are applied verbatim; the corpus requires no code change beyond document loading, and no model is trained.

Three observations follow (Table 8). First, the n-gram ordering inverts as the framework predicts. BSARD questions are legal questions of a dozen tokens rather than multi-thousand-token case descriptions, so phrase n-grams starve: the 5-gram configuration that leads precedent retrieval on IL-PCSR collapses to 0.83 F1, while the unigram lexical leg and the dense leg compete on near-equal terms. The rules are query-conditional, not corpus-bound. Second, the measured mechanisms transfer individually: prepending the act-context description to indexed article text, the same device that resolved act-name ambiguity in the expanded corpus, lifts unigram MAP from 13.20 to 15.14, and reciprocal rank fusion over the lexical and dense legs lifts recall at depth 100 from 51.65 to 54.62. Third, the assembled configuration exceeds every zero-shot baseline published for the benchmark, including its best dense bi-encoder row at 49.41 and its own BM25 row at 40.13; only the fine-tuned models score higher, at 74.78, reproducing the supervision headroom observed on IL-PCSR. The published rows are macro-averaged over queries while ours are micro-averaged, a mild protocol difference that changes no ranking here.

Table 8: External validation on BSARD (222 test questions, 22,633 French articles). This work reports micro-averaged metrics at depth 100; published rows report the paper's macro-averaged R@100.

Configuration	R@10	R@100	MAP@100	MRR@100
BM25, unigram, context-prepend	14.81	41.23	17.31	25.73
Qwen3-Embedding-0.6B	22.71	51.65	26.71	39.35
Qwen3-Embedding-4B	29.59	**67.67**	**36.91**	**51.20**
RRF, BM25 + Qwen3-0.6B (k=60)	22.08	54.62	25.88	37.62
RRF, BM25 + Qwen3-4B (k=60)	25.09	64.91	29.81	41.45
Published baselines (macro R@100)				
BM25, zero-shot		40.13		
Bi-encoder word2vec, zero-shot		49.41		
Fine-tuned, best		74.78		

The external dense runs also test the parameter hypothesis of Chapter 4, and the pattern repeats at larger amplitude. The 4B embedder lifts recall at depth 100 from 51.65 to 67.67 and MAP from 26.71 to 36.91 over the 0.6B variant on this corpus, a gain larger than the one measured between the two variants on IL-PCSR statutes, and the 4B leg alone closes most of the gap to the published fine-tuned ceiling of 74.78 without any task-specific training. Fusion behaviour sharpens the earlier finding in the same stroke: reciprocal rank fusion helps when the two legs carry complementary evidence, lifting the 0.6B configuration from 51.65 to 54.62, but dilutes a dominant leg, since the weak BM25 leg drags the 4B ranking from 67.67 down to 64.91. Fusion is beneficial only when the legs are competitive.

3.14 Synthesis of Findings

The measurements reduce to five design rules.

1. Statute retrieval is a semantic task: fuse 5-gram BM25 with a long-context embedder and rerank listwise.

2. Precedent retrieval is a lexical task: run whole-document 5-gram BM25 alone and rerank nothing.

3. Long case descriptions are issued verbatim; every compression mechanism measured loses ground.

4. No second stage may be used whose context window is smaller than the query.

5. Retrieval quality and answer reliability are different properties; the first is optimised in this chapter, the second is enforced by the agent loop of the next.

Chapter 4

Design and Architecture of the Agentic RAG Framework

4.1 From Findings to Architecture

Each element of the framework exists because a measurement in Chapter 3 placed it there; Table 9 states the mapping explicitly.

Table 9: Mapping of experimental findings to architectural decisions.

Measured finding	Architectural rule
5-gram BM25 dominates PCR outright	Precedent leg runs lexical retrieval alone, whole-document index
Dense and fused legs dominate LSR	Statute leg fuses lexical and long-context dense legs by RRF
Listwise rerank helps LSR, hurts PCR	Reranker sits on the statute leg only
Cross-encoder collapses recall	No context-bounded reranker in pipeline
All query compression fails	Case text issued verbatim to both legs
Agent planning harms retrieval; synthesis and validation are the value	LLM confined to reflect, retry, synthesise, validate; routing removed
Corpus lacks governing law for real queries	Statute corpus expanded; persistent insufficiency routes to labelled fallback

4.2 System Overview

The framework comprises three layers. The *evidence layer* is the retrieval substrate: two corpus legs plus the fusion and reranking stages that the statute leg requires. The *control layer* is the agentic loop: a bounded controller that judges evidence sufficiency, recovers once, synthesises the dossier, and validates citations. The *presentation layer* is the serving application: the API and browser interface through which the measured pipeline is exercised. Figure 1 shows the assembled system.

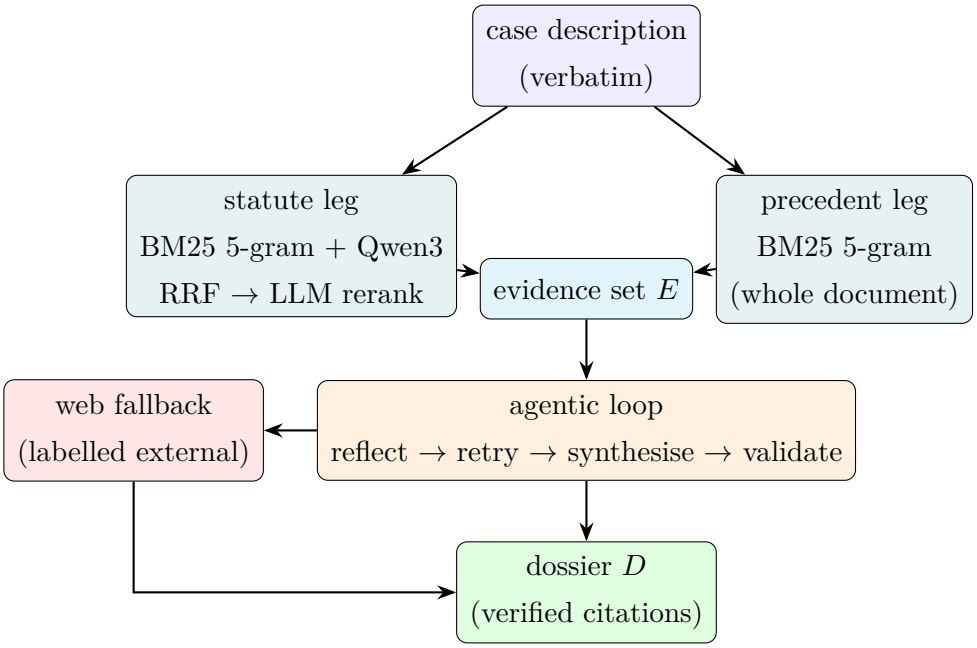


Figure 1: System architecture. The evidence layer retrieves through task-conditional legs; the control layer judges, recovers, synthesises, and validates; the fallback produces labelled external material only on persistent insufficiency.

4.3 The Retrieval Substrate

4.3.1 Document Representation

Precedent documents extend to tens of thousands of tokens and are indexed whole where the lexical leg requires document-spanning phrase context, and by recursive

splitting at a 512-token window with 50-token overlap where chunk-level retrieval is configured; Chapter 3 showed the choice to be second-order for BM25. Expanded-corpus statutes arrive already segmented at section boundaries. Each chunk records its parent document identifier, and retrieval deduplicates chunk hits to document hits so that evidence is always reported at document granularity.

4.3.2 The Lexical Leg

The lexical leg indexes documents as word n-grams under BM25S with $k_1 = 1.6$, $b = 0.7$. Statutory and precedential language is phrasal, and phrase matching is the source of this leg's strength: on precedent retrieval the 5-gram variant is the strongest non-fine-tuned configuration measured by any mechanism in this study. The leg is deterministic, auditable, and cache-addressed.

4.3.3 The Dense Leg

The dense leg embeds chunks with an instruction-prompted bi-encoder and retrieves by inner product over normalised vectors. Corpus embeddings are cached to disk so encoding is a one-time cost; per-query encoding adds sub-second latency. The serving configuration employs the 0.6B Qwen3-Embedding variant. We hypothesise that increasing model parameters and context length improves retrieval performance, as demonstrated by the gain from the 0.6B to the 4B variant in Table 1 and confirmed externally on BSARD, where the 4B leg lifts R@100 from 51.65 to 67.67 (Table 8).

4.3.4 Fusion and Reranking

Where both legs operate, reciprocal rank fusion merges them at $k = 60$ over each leg's top ranks. On the statute leg the fused pool is then reranked listwise: the twenty strongest candidates are presented as numbered snippets and the model returns a relevance ordering, with the fused order preserved unchanged upon any failure. The precedent leg omits the dense component and the reranker entirely.

4.4 The Agentic Control Loop

The controller is a bounded state machine over the evidence set E: retrieve, reflect, optionally retry once, synthesise, validate. Figure 2 shows the flow, and each state is specified below.

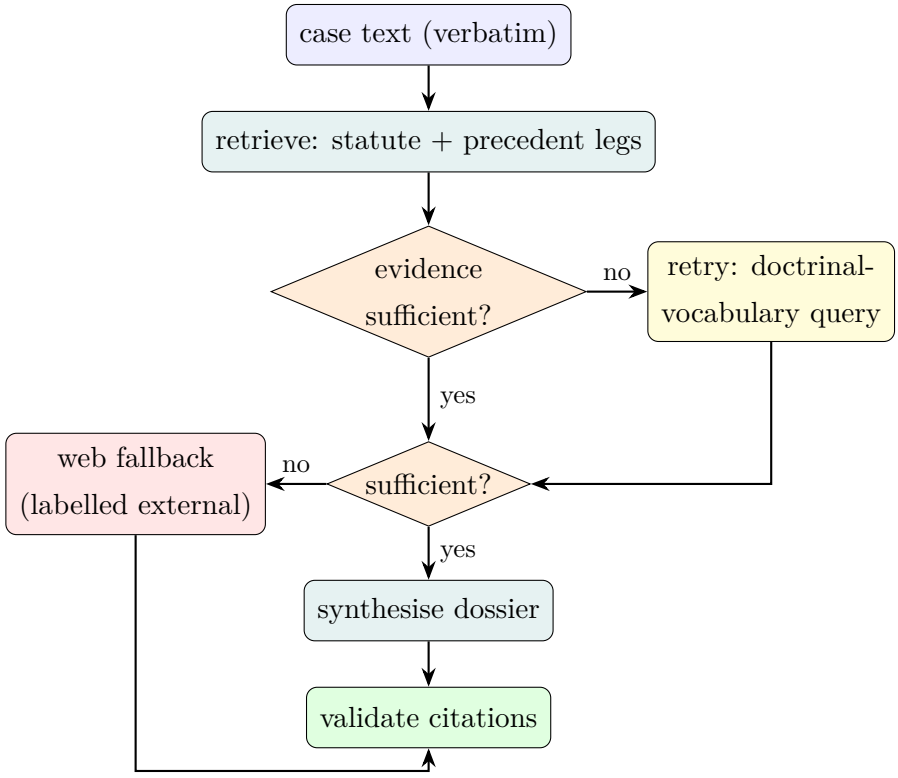


Figure 2: The agentic loop. Reflection judges evidence semantically on every query; a single reformulated retry executes upon a negative verdict; residual gaps route to a labelled external fallback.

Retrieve. The case text is issued verbatim to both legs. Chapter 3 establishes that every alternative loses.

Reflect. The language model examines the evidence and determines whether it genuinely addresses the case. The judgment is semantic rather than score-based, because a lexical score cannot distinguish the correct statute from a similarly named incorrect one: sections of the Bharatiya Nyaya Sanhita, the Bharatiya

Nagarik Suraksha Sanhita, and the Bharatiya Sakshya Adhiniyam share much of their vocabulary and score comparably while only one is germane. Upon a negative verdict the model additionally formulates refined queries in doctrinal vocabulary, the phrasing statutory text itself employs, because bare section numbers do not occur in section bodies.

Retry. A negative verdict triggers one further retrieval round using the refined queries, and the merged evidence is judged once more. The loop is bounded at a single retry, since a lexical retriever that has failed twice rarely surfaces new material on a third attempt.

Synthesise. The model produces a structured dossier from the final evidence: a direct answer, the applicable provisions, a grounded analysis, and recommended steps. The prompt requires the answer to state what the cited provisions establish and prohibits commentary about unused or absent documents; where the evidence cannot support an answer, the dossier states so once rather than fabricating authority.

Validate. Every citation in the dossier is intersected with the retrieved set, and unverifiable identifiers are discarded with a warning. The check is mechanical and admits no exceptions.

Fallback. If the post-retry verdict remains negative, the residual gap is answered through web search, and the result is labelled as not corpus-grounded. External material may inform the answer; it is never presented as retrieved authority.

Algorithm 1 Agentic control loop

Require: case description q

$E \leftarrow \textsc{Retrieve}(q)$ ▷ verbatim to both legs
$v \leftarrow \textsc{Reflect}(q, E)$ ▷ semantic verdict
if $v =$ insufficient **then**
    $q' \leftarrow \textsc{Reformulate}(q, E)$ ▷ doctrinal vocabulary
    $E \leftarrow E \cup \textsc{Retrieve}(q')$ ▷ bounded retry
    $v \leftarrow \textsc{Reflect}(q, E)$
end if
if $v =$ insufficient **then**
    $F \leftarrow \textsc{WebFallback}(q)$ ▷ labelled external
else
    $F \leftarrow \emptyset$
end if
$D \leftarrow \textsc{Synthesise}(q, E \cup F)$
$\textsc{Validate}(D, E)$ ▷ discard unverifiable citations
return D

4.5 Expanded Statutory Corpus

The benchmark statute collection is bounded by design; real queries invoke legislation it does not contain, including the 2023 criminal codes, the Digital Personal Data Protection Act, the Indian Easements Act, and the Insolvency and Bankruptcy Code. The statute leg is therefore extended with a corpus assembled from two public sources: a collection of Indian central acts in India Code markdown [42] and the KanoonGPT legal documents corpus [43]. Approximately 300 central statutes were selected across criminal, civil and property, family, consumer, finance and insolvency, labour, environment, health, governance, intellectual property, and digital regulation. Each act is segmented at section markers into one document per section, and the act title is prepended to each section's indexed text. This last detail is load-bearing: section bodies never contain their act's name, so an act-name query

cannot lexically distinguish the Bharatiya Nyaya Sanhita from the similarly named procedural and evidence codes unless the title is indexed. Candidate source texts are scored by English legal-term density to reject corrupted encodings.

Table 10: Composition of the expanded statute index. The precedent leg retains its 3,183 IL-PCSR documents.

Corpus segment	Section documents
Criminal codes and procedure	1,027
Finance, insolvency, securities, taxation	3,307
Civil procedure, property, succession	2,556
Labour codes and industrial law	1,571
Family and personal law	1,224
Health, food, mental healthcare	2,318
Environment, forests, energy	1,204
Consumer, competition, commerce	1,087
Governance, transparency, public services	1,612
Digital regulation (IT Act, DPDP, telecom)	517
Intellectual property	1,296
Constitutional and administrative law	1,846
Other central acts	9,573
Expanded corpus (302 acts)	27,138
IL-PCSR statutes (retained)	936
Indexed statute leg	28,074

Chapter 5

Implementation and Application Development

5.1 Technology Stack

The framework is implemented in Python. Retrieval uses BM25S for the lexical legs [19], sentence-transformers for dense encoding, and NumPy matrix products for similarity search; the agent and reranker call an externally hosted language model through a plain HTTP provider layer. The experiment harness and the server share the same loader, chunker, retriever classes, and evaluation code, which keeps the served pipeline identical to the benchmarked one. Heavy computation is offloaded: dense corpus encoding ran on a Kaggle T4 and a Modal A10G GPU, while the serving machine only loads the cached embedding matrix. The server is a FastAPI application; the demonstration interface is a React single-page application.

5.2 The Evaluation Harness

The harness is configuration-driven. Each experiment is a JSON specification naming its chunking strategy, retriever, query strategy, and reranker; the runner indexes once per corpus (content-addressed, so an index is rebuilt only when corpus bytes change), issues all 627 queries through one code path, and writes both an aggregate

summary and a per-query record containing the run's git commit, config hash, seed, and platform.

5.3 The Agent Implementation

The controller is implemented as a single orchestrator class with explicit steps. The retrieval step merges hits per task into an evidence map keyed by document identifier, keeping the best score per document across rounds. The reflection step presents the top evidence identifiers and titles to the model and parses a JSON verdict; unparseable or failed verdicts fail open as sufficient, so an LLM outage cannot mark good evidence insufficient. The synthesis step assembles the top-k evidence snippets with their identifiers and titles into the prompt and parses the returned JSON dossier. The validation step intersects the dossier's citation identifiers with the evidence map and drops any that do not occur there. Each step records its inputs and latency into a trace that the application renders, so every agent decision is inspectable.

5.4 The Serving Application

The server exposes three analysis modes that differ honestly in what they run. *Single-shot* issues the case text to both first-stage legs and returns raw evidence with no LLM call, which makes it the free baseline and the retrieval inspection view. *Agent-synthesis* runs the recommended configuration: verbatim retrieval, then grounded dossier generation with citation validation. *Agent-full* executes the complete loop: retrieve, reflect, bounded retry, synthesise, validate, and a labelled web fallback on persistent insufficiency. The statute leg is assembled at startup as the measured configuration permits: lexical retrieval alone when no dense embedding cache is present, RRF fusion when the cache exists, and listwise reranking on the agent paths when an LLM key is configured. Each response carries the leg labels actually used, so the interface cannot misrepresent the active pipeline. Conversations persist to a server-side store with search, pin, rename, and delete operations,

and each analysis is deep-linkable by identifier.

The interface renders the dossier first, then the underlying evidence as inspectable cards: each source carries its corpus, retrieval score, and a query-relevant excerpt chosen by content-term overlap with the case text, so the grounding of every answer is verified by inspection rather than asserted. External fallback material appears in a separately labelled section.

5.5 End-to-End Request Flow

A case description enters the API; the server issues it verbatim to both legs. The statute leg retrieves through its fused, reranked pipeline; the precedent leg retrieves through lexical matching alone. The merged evidence set passes to reflection; on a sufficiency verdict, synthesis produces the dossier directly; on insufficiency, refined queries re-retrieve once, and a persistent failure routes the residual question to the labelled web fallback. Citation validation then filters the dossier before the response is stored and rendered. The full trace, including query counts, verdicts, and per-step latency, is returned with the answer.

Chapter 6

Results, Evaluation, and Discussion

6.1 Evaluation Framework

Three layers of evidence are kept distinct throughout this chapter: benchmark retrieval metrics (Chapter 3), agent behavioural statistics on the 50-query slice, and live verifications on the served system over the expanded corpus. Conflating them would produce claims the data does not support.

6.2 System-Level Behaviour

Table 11 summarises the assembled system as measured.

Table 11: The assembled system as measured. Retrieval rows are development-split; the test row is the test-split BM25 run.

Property	Measured value
Statute leg (dev)	RRF 5-gram + Qwen3: F1@10 14.68, MAP 19.12, MRR 40.53; + LLM rerank: F1@10 14.83, MAP 25.20, MRR 50.88
Precedent leg (dev)	5-gram BM25: F1@10 20.73, MAP 39.90, MRR 57.28
Test-split BM25 rec-512	F1@10 12.92 LSR / 17.92 PCR
Agent loop (50 q)	78 retrieve calls; 22/50 sufficient at first verdict; 49/50 dossiers; 167/167 citations valid
Indexed corpora	28,074 statute docs; 3,183 precedent docs

6.3 Live Verifications

Five queries exercised against the served system illustrate the loop's operating modes.

Statute query: punishment for murder

Query. What is the punishment prescribed for murder under the Bharatiya Nyaya Sanhita, 2023?

Retrieved. BNS §§ 103, 104, 105; IPC § 302 (predecessor provision).

Loop. Evidence judged sufficient on the first pass; no retry.

Answer. "Section 103(1) of the Bharatiya Nyaya Sanhita, 2023 provides that whoever commits murder shall be punished with death or imprisonment for life and shall also be liable to fine. Under section 103(2), each member of a group of five or more acting in concert to commit murder …"

Case-law query: unlawful eviction

Query. A landlord forcibly evicted a tenant without notice and locked the premises; the tenant seeks relief for unlawful eviction and recovery of possession.

Retrieved. Specific Relief Act 1963 § 6; Delhi Rent Control Act 1958 §§ 14, 19, 22; four precedents.

Loop. First verdict insufficient; doctrinal retry.

Answer. "…a person dispossessed without consent and otherwise than in due course of law may bring a suit concerning that dispossession, and …a person entitled to

possession may seek recovery through the civil procedure provided by law."

General legal knowledge: culpable homicide and murder

Query. What is the difference between culpable homicide and murder under Indian criminal law?

Retrieved. IPC §§ 299, 300; BNS §§ 100, 101.

Loop. First verdict insufficient; doctrinal retry.

Answer. "Culpable homicide is the broader offence. Under Sections 299 IPC and 100 BNS, it is established where a person causes death with the specified intention or knowledge. Murder is the more serious, narrower category: under Sections 300 IPC and 101 BNS …"

Out-of-corpus: an international obligation

Query. What obligations does India bear under the Vienna Convention on Consular Relations for detained foreign nationals?

Retrieved. BNSS § 48; Immigration and Foreigners Act 2025; Extradition Act 1962; Foreigners Act 1946.

Loop. Insufficient, retried, then answered only from domestic provisions; the convention text is absent from the corpus, and no external material was presented as retrieved authority.

Answer. "The cited provisions establish domestic procedural duties upon arrest: officers must promptly notify a person nominated by the arrested individual, record whom they informed, and the Magistrate must check compliance …"

Out-of-scope query: integrity check

Query. Explain how Dijkstra's shortest-path algorithm works on a weighted graph.

Retrieved. Off-topic municipal and railway provisions; the corpus contains no relevant material.

Loop. Insufficient on both rounds.

Answer. "The evidence does not establish an answer." The loop declined to fabricate one.

The first box exhibits the clean path: sufficient evidence on the first pass, a corpus-grounded answer, no recovery. The second and third show the loop's characteristic behaviour: a retrieval failure invisible to lexical scores was caught by the

semantic verdict, and the doctrinal retry completed the correction inside the corpus, drawing on the expanded statute collection for the Delhi Rent Control provisions. The fourth box marks the corpus boundary honestly: the answer confines itself to the domestic provisions retrieved, and the missing convention text is neither invented nor passed off. The fifth states the property a legal tool cannot compromise: confronted with a query the corpus cannot answer, the system says so rather than synthesising authority that does not exist.

6.4 Interpretation

Four properties of the results deserve explicit discussion.

First, retrieval quality and answer reliability are different quantities, and the system optimises them in different places. The retrieval chapter maximises ranking metrics; the agent loop then guards the property metrics cannot express: whether the evidence actually answers the question and whether every citation is real. The 28 negative first verdicts in the agent benchmark are the loop working as designed, not a defect rate.

Second, the asymmetric architecture is not a compromise but the measured optimum. A uniform pipeline would either give statutes a lexical leg twenty MAP points below its dense alternative, or give precedents a fused leg that dilutes the strongest non-fine-tuned configuration in the study.

Third, the agent's contribution is located where the measurements place it. The router arm of the agent benchmark shows that LLM-written retrieval queries lose nearly twenty MAP points on the lexical task; what the LLM contributes is judgment, recovery, synthesis, and citation discipline, and the architecture confines it to exactly those functions.

Fourth, the negative results are load-bearing. The study can state with evidence, rather than with intuition, that long legal queries must be issued verbatim, that bounded-context rerankers must not be used on them, and that fusion must be conditioned on leg quality. These rules generalise beyond the specific configuration measured.

6.5 Limitations and Threats to Validity

Development-split metrics at fixed depth are reported alongside, not equated with, the published test-split best-k rows; a same-protocol evaluation of the assembled configuration on the test split remains outstanding, and the single test-split run performed covers lexical retrieval only. The 4B embedder that marks the measured quality ceiling requires GPU inference; the serving configuration employs the 0.6B variant and accepts the corresponding gap. Queries to dense models were truncated at 2,048 tokens, so the dense legs were measured on roughly half the average case description; the results favour them nonetheless. The dense legs evaluated on the precedent task used chunked indexes, while the champion precedent row used a whole-document index, a pairing fixed by n-gram tokenisation cost rather than by choice. Precedent coverage is bounded by the benchmark's 3,183 documents, and queries beyond it rely on the labelled fallback. The retry is bounded at one round by design; a learned stopping criterion could recover a small residue of failures. Finally, the agent benchmark is 50 queries on one model configuration; the behavioural findings are consistent but their generality across models is unmeasured.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This dissertation set out to determine which pipeline a verifiable legal-analysis system should run, and the measurements answered the question consistently. The two retrieval tasks are distinct problems warranting distinct machinery: phrase-level lexical retrieval for precedents, long-context dense retrieval with fusion and listwise reranking for statutes. Long case descriptions must reach the retriever verbatim; every substitute tested lost ground. A reranker is warranted only where the relevance signal is semantic and only when it can read the input. An agentic loop earns its place through the guarantees it adds, namely evidence-sufficiency judgment, a bounded retry, a labelled fallback, and universally validated citations, rather than through retrieval planning. A first external validation on the French BSARD benchmark confirms that these design rules travel: without modification or retraining, the pipeline exceeds every published zero-shot baseline, and the 4B dense leg alone closes most of the remaining gap to the fine-tuned ceiling. The expanded corpus and the demonstration application render the measured pipeline a functioning system that answers queries the benchmark itself cannot, including the 2023 criminal codes.

7.2 Findings in Relation to the Research Objectives

The first two objectives are met by the experimental programme: Tables 1 and 2 establish which retrieval, fusion, and reranking mechanisms are effective for each corpus, and Table 9 records how those measurements, including the uniformly negative query-processing results of Table 4, determined the derived architecture. The third objective is met by the agent benchmark: the bounded loop contributed sufficiency judgment, one-shot recovery, and citation integrity, with all 167 emitted citations valid. The fourth is met by the live verifications: the expanded corpus resolves queries on legislation the benchmark does not contain. The fifth objective, transfer to other legal datasets and jurisdictions, receives its first empirical test in Section 3.13: on the French BSARD benchmark the assembled legs outperform every published zero-shot baseline without modification or retraining, while broader transfer across further corpora remains open.

7.3 Future Work

A same-protocol test-split evaluation of the assembled configuration would convert the protocol-aware comparison into a direct numerical one, and is the first outstanding item. Fine-tuning the dense leg on the 5,017 training queries constitutes the largest unclaimed improvement, since the published fine-tuned rows indicate the headroom supervision provides; a contrastive fine-tune of Qwen3-0.6B is configured but not yet evaluated. A task-aware precedent reranking prompt, a learning-to-rank fusion trained on per-leg features, and cross-task conditioned retrieval are all specified extensions the harness supports directly. On the corpus side, state-level legislation, municipal bye-laws, and state police acts are the next class of statutes real queries invoke, and the ingestion pipeline requires no modification to accept them. On the agent side, a learned sufficiency threshold, a second bounded retry, and a numeric groundedness score per dossier are natural strengthenings. Finally, the fifth objective stands partially open in empirical terms: Section 3.13 supplies

the first cross-corpus evidence on French statutory text, and extending the same battery to the case-law and non-European benchmarks the literature supplies, the COLIEE Canadian case-law collection [44] and the Chinese case-retrieval corpora LeCaRD [45] and CAIL [46], is the outstanding test. The evaluation itself can be deepened: a human study of dossier correctness against expert legal judgment would measure the property the citation validator can only approximate.

References

[1] Shounak Paul, Dhananjay Ghumare, Pawan Goyal, Saptarshi Ghosh, and Ashutosh Modi. IL-PCSR: Legal corpus for prior case and statute retrieval. arXiv:2511.00268, 2025. Findings of EMNLP 2025, IIT Kharagpur and IIT Kanpur.

[2] Neel Guha, Julian Nyarko, Daniel E. Ho, Christopher Ré, et al. LegalBench: A collaboratively built benchmark for measuring legal reasoning in large language models. *Advances in Neural Information Processing Systems (NeurIPS), Datasets and Benchmarks Track*, 2023.

[3] Atharva Joshi, Abhay Shukla, Ananya Pratap Singh, Paheli Bhattacharya, Saptarshi Ghosh, Kripabandhu Ghosh, and Ashutosh Modi. IL-TUR: Benchmark for Indian legal text understanding and reasoning. arXiv:2407.10361, 2024.

[4] Chaojun Xiao, Xueyu Hu, Zhiyuan Liu, Cunchao Tu, and Maosong Sun. Lawformer: A pre-trained language model for Chinese legal long documents. *AI Open*, 2:79–84, 2021.

[5] Haitao Li, Qingyao Ai, Jia Chen, Qian Dong, Yueyue Wu, Yiqun Liu, Chong Chen, and Qi Tian. SAILER: Structure-aware pre-trained language model for legal case retrieval. In *Proceedings of the 46th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1035–1044, 2023.

[6] National Judicial Data Grid. Pending cases in Indian district and taluka courts.

`https://njdg.ecourts.gov.in`, 2025. Department of Justice, Government of India; accessed 2025.

[7] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 33, pages 9459–9474, 2020.

[8] Gerard Salton, Anita Wong, and Chung-Shu Yang. A vector space model for automatic indexing. *Communications of the ACM*, 18(11):613–620, 1975.

[9] Gerard Salton and Christopher Buckley. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, 24(5):513–523, 1988.

[10] Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval*, 3(4):333–389, 2009.

[11] Bingfeng Luo, Yansong Feng, Jianbo Xu, Xiang Zhang, and Dongyan Zhao. Learning to predict charges for criminal cases with legal basis. In *Proceedings of the 2017 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 2727–2736, 2017.

[12] Haoxi Zhong, Zhipeng Guo, Cunchao Tu, Chaojun Xiao, Zhiyuan Liu, and Maosong Sun. Legal judgment prediction via topological learning. In *Proceedings of the 2018 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 3540–3549, 2018.

[13] Shounak Paul, Pawan Goyal, and Saptarshi Ghosh. LeSICiN: A heterogeneous graph-based approach for automatic legal statute identification from Indian legal documents. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 11139–11146, 2022.

[14] Shounak Paul, Rajas Bhatt, Pawan Goyal, and Saptarshi Ghosh. Legal statute identification: A case study using state-of-the-art datasets and methods. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2231–2240, 2024.

[15] Yi Feng, Chuanyi Li, and Vincent Ng. Legal case retrieval: A survey of the state of the art. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 6472–6485, 2024.

[16] Vijit Malik, Rishabh Sanjay, Shubham Kumar Nigam, Kripabandhu Ghosh, Shouvik Kumar Guha, Arnab Bhattacharya, and Ashutosh Modi. ILDC for CJPE: Indian legal documents corpus for court judgment prediction and explanation. In *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing (ACL-IJCNLP)*, pages 4046–4062, 2021.

[17] Vijit Malik, Rishabh Sanjay, Shouvik Kumar Guha, Angshuman Hazarika, Shubham Nigam, Arnab Bhattacharya, and Ashutosh Modi. Semantic segmentation of legal documents via rhetorical roles. In *Proceedings of the Natural Legal Language Processing Workshop (NLLP) at EMNLP*, pages 153–167, 2022.

[18] Paheli Bhattacharya, Shounak Paul, Kripabandhu Ghosh, Saptarshi Ghosh, and Adam Wyner. DeepRhole: Deep learning for rhetorical role labeling of sentences in legal case documents. *Artificial Intelligence and Law*, 31:1–38, 2023.

[19] Xing Han Lù. BM25S: Orders of magnitude faster BM25 via sparse matrix operations. arXiv:2407.03618, 2024.

[20] Vladimir Karpukhin, Barlas Oğuz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. Dense passage retrieval for open-domain question answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pages 6769–6781, 2020.

[21] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, pages 3982–3992, 2019.

[22] Yanzhao Zhang, Mingxin Li, Dingkun Long, Xin Zhang, Huan Lin, Baosong Yang, Pengjun Xie, An Yang, Dayiheng Liu, Junyang Lin, Fei Huang, and Jingren Zhou. Qwen3 embedding: Advancing text embedding and reranking through foundation models. arXiv:2506.05176, 2025.

[23] Gordon V. Cormack, Charles L. A. Clarke, and Stefan Buettcher. Reciprocal rank fusion outperforms condorcet and individual rank learning methods. In *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 758–759, 2009.

[24] Sebastian Bruch, Siyu Gai, and Amir Ingber. An analysis of fusion functions for hybrid retrieval. *ACM Transactions on Information Systems*, 42(1):1–35, 2023.

[25] Rodrigo Nogueira and Kyunghyun Cho. Passage re-ranking with BERT. arXiv:1901.04085, 2019.

[26] Omar Khattab and Matei Zaharia. ColBERT: Efficient and effective passage search via contextualized late interaction over BERT. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 39–48, 2020.

[27] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. SPLADE: Sparse lexical and expansion model for first stage ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2288–2292, 2021.

[28] Jianlv Chen, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian, and Zheng Liu.

BGE M3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation. arXiv:2402.03216, 2024.

[29] Shengjie Ma, Chong Chen, Qi Chu, and Jiaxin Mao. Leveraging large language models for relevance judgments in legal case retrieval. *arXiv preprint arXiv:2403.18405*, 2024.

[30] J. J. Rocchio. Relevance feedback in information retrieval. Technical report, Salton's SMART Retrieval System: Experiments in Automatic Document Processing, Prentice-Hall, 1971.

[31] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023.

[32] Aditi Singh, Abul Ehtesham, Saket Kumar, and Tala Talaei Khoei. Agentic retrieval-augmented generation: A survey on agentic RAG. arXiv:2501.09136, 2025.

[33] Akari Asai, Zeqiu Wu, Yizhong Wang, Avirup Sil, and Hannaneh Hajishirzi. Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In *International Conference on Learning Representations (ICLR)*, 2024.

[34] Abhinav Joshi, Akshat Sharma, Sai Kiran Tanikella, and Ashutosh Modi. U-CREAT: Unsupervised case retrieval using events extrAcTion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, pages 13899–13915, 2023.

[35] Yanran Tang, Ruihong Qiu, Yilun Liu, Xue Li, and Zi Huang. CaseGNN: Graph neural networks for legal case retrieval with text-attributed graphs. In *Advances in Information Retrieval (ECIR)*, pages 80–95, 2024.

[36] Yanran Tang, Ruihong Qiu, Hongzhi Yin, Xue Li, and Zi Huang. CaseLink: Inductive graph learning for legal case retrieval. In *Proceedings of the 47th*

International ACM SIGIR Conference on Research and Development in Information Retrieval, pages 2199–2209, 2024.

[37] Paheli Bhattacharya, Kripabandhu Ghosh, Arindam Pal, and Saptarshi Ghosh. Hier-SPCNet: A legal statute hierarchy-based heterogeneous network for computing legal case document similarity. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 1657–1660, 2020.

[38] Weicong Qin, Zelin Cao, Weijie Yu, Zihua Si, Sirui Chen, and Jun Xu. Explicitly integrating judgment prediction with legal document retrieval: A law-guided generative approach. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2210–2220, 2024.

[39] Prathamesh Kalamkar, Aman Tiwari, Astha Agarwal, Saurabh Karn, Smita Gupta, Vivek Raghavan, and Ashutosh Modi. Corpus for automatic structuring of legal documents. In *Proceedings of the 13th Language Resources and Evaluation Conference (LREC)*, pages 4420–4429, 2022.

[40] Prathamesh Kalamkar, Astha Agarwal, Aman Tiwari, Smita Gupta, Saurabh Karn, and Vivek Raghavan. Named entity recognition in Indian court judgments. *arXiv preprint arXiv:2211.03442*, 2022.

[41] Antoine Louis and Gerasimos Spanakis. A statutory article retrieval dataset in French. In *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Long Papers)*, pages 6789–6803, 2022.

[42] geekyrakshit. Indian Legal Acts dataset. `https://huggingface.co/datasets/geekyrakshit/indian-legal-acts`, 2025. 883 Central Acts with India Code Markdown; accessed 2025.

[43] KanoonGPT. Indian Legal Documents dataset. `https://huggingface.co/datasets/KanoonGPT/indian-legal-documents`, 2025. 35,845 typed legal documents; accessed 2025.

[44] Juliano Rabelo, Randy Goebel, Mi-Young Kim, Yoshinobu Kano, Masaharu Yoshioka, and Ken Satoh. Overview and discussion of the competition on legal information extraction/entailment (COLIEE) 2021. *The Review of Socionetwork Strategies*, 16(1):111–133, 2022.

[45] Yixiao Ma, Yunqiu Shao, Yueyue Wu, Yiqun Liu, Ruizhe Zhang, Min Zhang, and Shaoping Ma. LeCaRD: A legal case retrieval dataset for Chinese law system. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, pages 2342–2348, 2021.

[46] Chaojun Xiao, Haoxi Zhong, Zhipeng Guo, Cunchao Tu, Zhiyuan Liu, Maosong Sun, Yansong Feng, Xianpei Han, Zhen Hu, Heng Wang, and Jianfeng Xu. CAIL2018: A large-scale legal dataset for judgment prediction. *arXiv preprint arXiv:1807.02478*, 2018.